

EE708: Fundamentals of Data Science and Machine Intelligence

Rajesh M Hegde
Dept. of EE IIT Kanpur

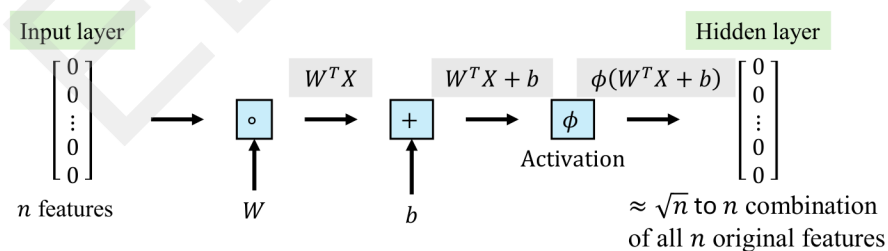


Module 7B Convolutional Neural Networks (CNN)

1

Feature Extraction and Representation Learning

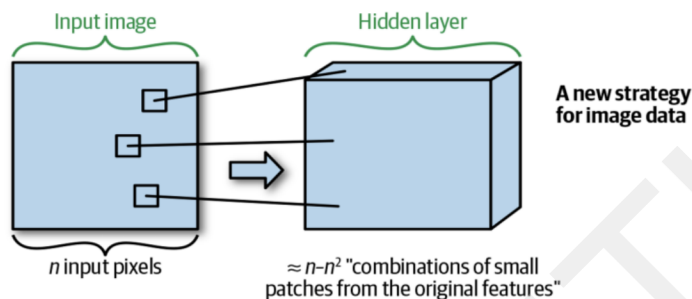
- Neural networks automatically discover important feature combinations during training.
- Training starts with random feature combinations using a random weight matrix.
- Neural networks refine useful combinations and discard irrelevant ones.
- The process of identifying essential feature combinations is called *representation learning*.



The neural networks start with n features and then learn somewhere between $\sqrt{n}$ and n "combinations" of these features to make predictions

Feature Extraction and Combinations for Image Data

- The solution, at a high level, will be to create combinations of features
- Can be of an order of magnitude more - each one be a combination of the pixels from a small rectangular patch in the input image.



With image data, we can define each learned feature to be a function of a small patch of data, and thus define somewhere between n and n^2 output neurons

EE708

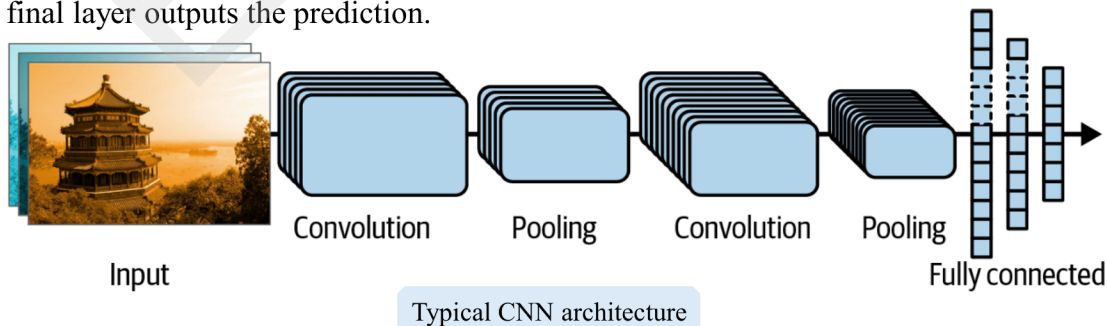
Module 7

3

3

Convolutional Neural Network Architecture

- Typical CNN architectures stack a few convolutional layers.
- Each convolution layer is generally followed by a ReLU layer, then a pooling layer
- The image gets smaller and smaller as it progresses through the network, but it also typically gets deeper and deeper (i.e., with more feature maps).
- At the top of the stack, a regular feedforward NN is added, composed of a few fully connected layers (with ReLUs).
- The final layer outputs the prediction.



EE708

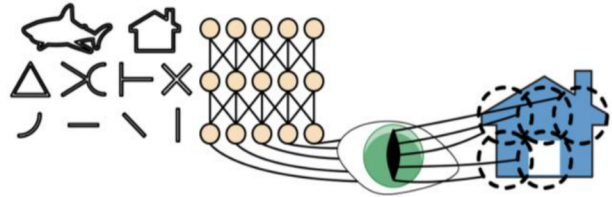
Typical CNN architecture

4

4

CNN : The Architecture of the Visual Cortex

- *David H. Hubel* and *Torsten Wiesel* performed a series of experiments on cats in 1958 and 1959 (later on monkeys).
- Giving crucial insights into the structure of the visual cortex (received the Nobel Prize in Physiology or Medicine in 1981).



The local receptive fields of five neurons are represented by dashed circles.

- The study showed that many neurons in the visual cortex have a small local receptive field, that is, they react only to visual stimuli located in a limited region of the visual field.
- Overlapping receptive fields collectively cover the entire visual field.
- Higher-level neurons process outputs from neighboring lower-level neurons.
- This architecture enables the detection of complex patterns across the visual field.

EE708

Module 7

5

5

The Architecture of the Visual Cortex

- Studies of the visual cortex inspired the **Neocognition** (introduced in 1980), which gradually evolved into **convolutional neural networks**.
- An important milestone was a paper by *Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner (1998)*, which introduced the famous **LeNet-5 architecture**, widely used to **recognize handwritten check numbers**.
 - This architecture has some existing building blocks, such as fully connected layers and sigmoid activation functions, but it also introduces **two new building blocks: convolutional layers and pooling layers**.

EE708

Module 7

6

6

Why CNN ?

Why not simply use a regular deep neural network with fully connected layers for image recognition tasks?

- DNN works fine for small images (e.g., MNIST).
- However, breaks down for larger images because of the huge number of parameters it requires.
- For example: 100×100 pixel image has 10,000 pixels, and if the first layer has just 1,000 neurons (which already severely restricts the amount of information transmitted to the next layer), this means a total of 10 million connections. And that's just the first layer.
- CNNs solve this problem using partially connected layers and weight sharing.

EE708

Module 7

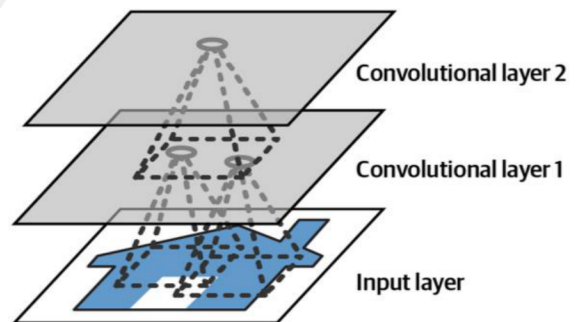
7

7

CNN : Convolutional Layers

The most important building block of a CNN is the convolutional layer

- Neurons in the first convolutional layer are connected only to pixels in their receptive fields.
- Each neuron in the second convolutional layer is connected only to neurons located within a small rectangle in the first layer.



CNN layers with rectangular local receptive fields

- This allows the network to concentrate on small low-level features in the first hidden layer, then assemble them into larger higher-level features in the next hidden layer, and so on.
- This hierarchical structure is common in real-world images, which is one of the reasons why CNNs work so well for image recognition.

EE708

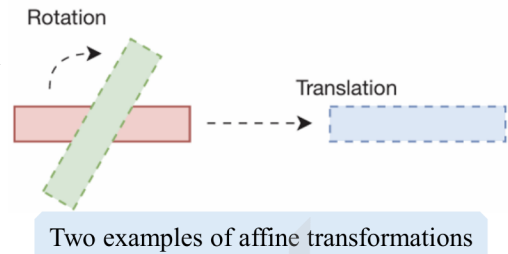
Module 7

8

8

CNN : Translation Invariant

- If the network is mostly trained with pictures of cats in the middle of the image, a convolutional network can still be able to classify the image as “containing a cat” when presented with an image with a cat in one of the corners.
- Translational invariance is achieved by employing weight sharing between neurons and making them sparsely connected.
- Translation is a geometric transformation known as an *affine transformation*.
 - It changes the location of an object without changing its shape.
 - Another common affine transformation is rotation, which changes the orientation of an object.



EE708

Module 7

9

9

The Convolution Operation in CNN

- **Image Feature** : is a combination of pixels from a local patch of an image.
- Consider a 3×3 pixel image with each pixel labeled with a letter:
- Let us calculate a new feature that is a function of the 2×2 patch of pixels.
- For that consider a 2×2 kernel with each weight labeled with a Greek letter:
- Define new features as linear combinations of old features.
- Since the filter fits in the image four times, output will be 2×2 image:

A	B	C
D	E	F
G	H	I

α	β
γ	δ

P	Q
R	S

EE708

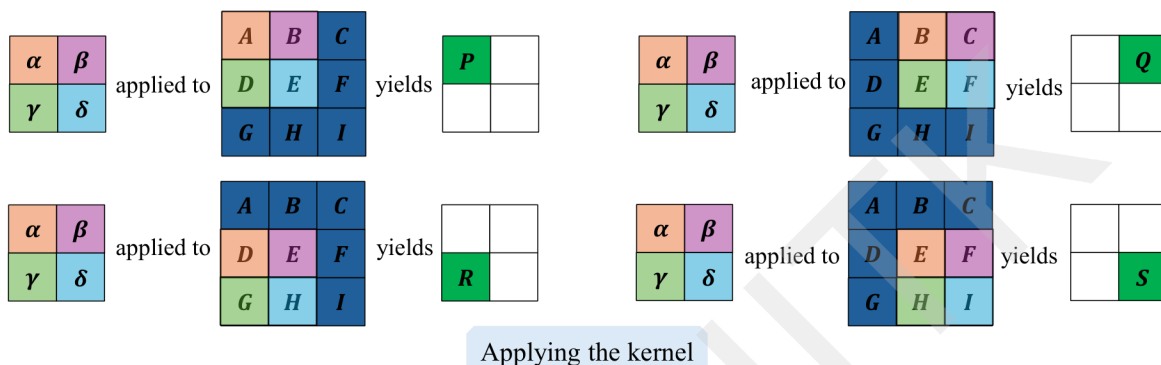
Module 7

10

10

The Convolution Operation in CNN

- Output is computed as the dot product of the weight and the corresponding image patch.
- The obtained value is then treated like the other computed features seen in neural networks:
- A bias may be added, followed by an activation function, producing a "neuron" or "learned feature" for the next network layers.



EE708

Module 7

11

11

The Convolution Operation : Equation View

Diagram showing the equation view of the convolution operation. It starts with a 3x3 input patch (values $A, B, C, D, E, F, G, H, I$) convolved with a 2x2 kernel (weights $\alpha, \beta, \gamma, \delta$) to give a 2x2 output feature map (values P, Q, R, S). Below this, four equations are shown, each representing the calculation of one output value P, Q, R, S . Each equation is the dot product of the kernel and the corresponding input patch, plus a bias term b .

$$\alpha \times A + \beta \times B + \gamma \times D + \delta \times E + b = P$$

$$\alpha \times B + \beta \times C + \gamma \times E + \delta \times F + b = Q$$

$$\alpha \times D + \beta \times E + \gamma \times G + \delta \times H + b = R$$

$$\alpha \times E + \beta \times F + \gamma \times H + \delta \times I + b = S$$

Equation view of the convolution

EE708

Module 7

12

12

The Convolution Operation : Matrix Multiplication

Convolution as a matrix multiplication

α	β	0	γ	δ	0	0	0	0
0	α	β	0	γ	δ	0	0	0
0	0	α	β	0	γ	δ	0	0
0	0	0	α	β	0	γ	δ	0

 $\times$

A
B
C
D
E
F
G
H
I

 $+$

b
b
b
b

 $=$

$\alpha A + \beta B + \gamma D + \delta E + b$
$\alpha B + \beta C + \gamma E + \delta F + b$
$\alpha D + \beta E + \gamma G + \delta H + b$
$\alpha E + \beta F + \gamma H + \delta J + b$

 $=$

P
Q
R
S

- Weight matrix is similar to ones from traditional neural networks.
- However, this weight matrix has two special properties:
 1. The zeros are untrainable, that is, they'll stay zero throughout the optimization process.
 2. Some of the weights are equal, and while they are trainable, they must remain equal.

EE708

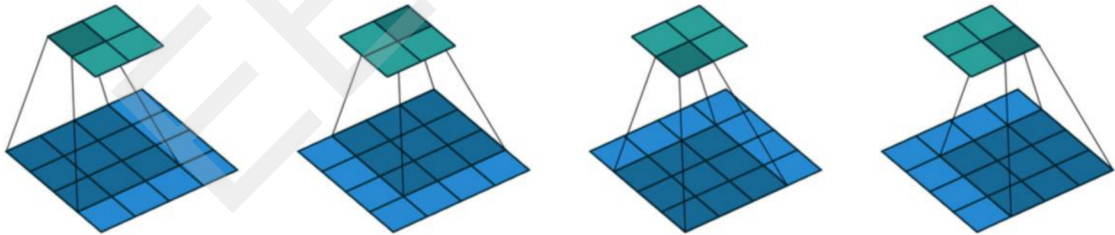
Module 7

13

13

Visualizing The Convolution Operation

- Example: A light blue 4×4 image with a dark blue 3×3 kernel applied to create a 2×2 green output activation map at the top.



Result of applying a 3x3 kernel to a 4x4 image (courtesy of Vincent Dumoulin and Francesco Visin)

EE708

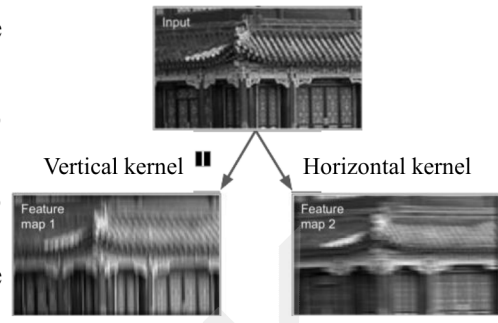
Module 7

14

14

The Convolution Operation : Vertical and Horizontal Kernels

- A neuron's weight corresponds to an image the size of its receptive field.
- Neurons with the same filter create a feature map, emphasizing areas most responsive to the filter.
- Kernels are learned automatically during training, not manually defined.
- Higher layers combine learned filters into more complex patterns.



	Vertical kernel	Horizontal kernel
Kernel description	black square with a vertical white line in the middle	black square with a horizontal white line in the middle
Function	ignore everything in their receptive field except for the central vertical line	ignore everything in their receptive field except for central horizontal line
Inference	vertical white lines get enhanced while the rest gets blurred	horizontal white lines get enhanced while the rest is blurred out

EE708

Module 7

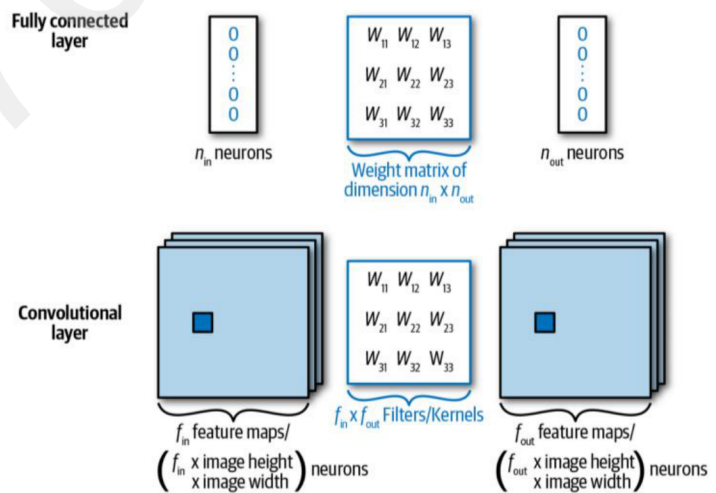
15

15

Difference Between Convolutional and Fully Connected Layers

Interpretation of individual neurons:

- Fully Connected Layer: Each neuron detects specific feature combinations learned from the prior layer in the entire observation.
- Convolutional Layer: Each neuron detects specific visual pattern combinations learned from the prior layer at a particular location in the input image.



Comparison between convolutional and fully connected layers

EE708

Module 7

16

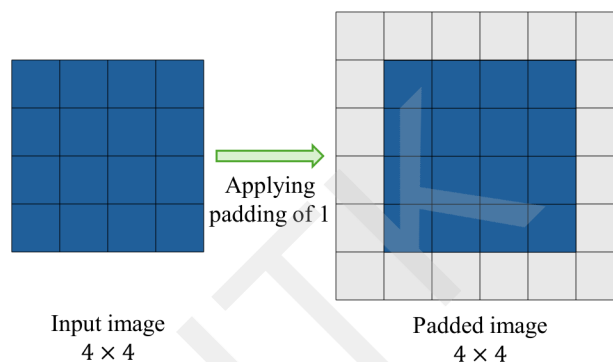
16

Convolution : Padding Zeros

- In the previous example, using 3×3 kernel on 4×4 image results in 2×2 output. The output is smaller than the input image.
- Padding refers to adding extra layers of pixels (usually zeros) around the input image before applying convolution.

- **Purpose:**

- To preserve spatial dimensions for deeper layers.
- To prevent the reduction of feature map size after each convolution.
- To allow border pixels to contribute equally to the output.



Padding an image

EE708

Module 7

17

17

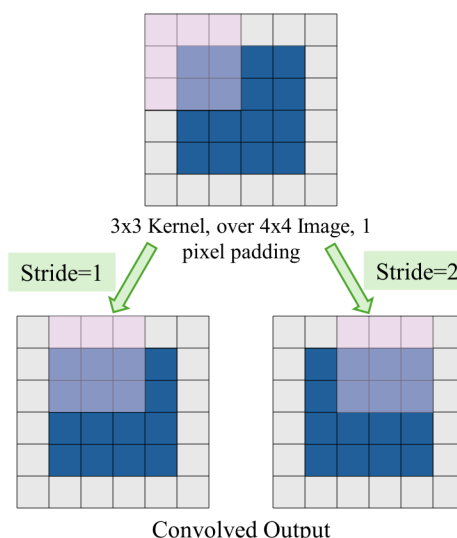
Convolution : Definition and Purpose of Stride

- **Stride** refers to the number of pixels by which the convolution filter moves (slides) across the input image during convolution.

- Stride = 1: Filter moves **one pixel** at a time.
- Stride > 1: Filter **skips pixels while moving**, reducing the overlap between regions and decreasing the output size.

- **Purpose:**

- To control the dimensions of the output feature map.
- To reduce overlapping regions in convolution operations for distinct feature detection.



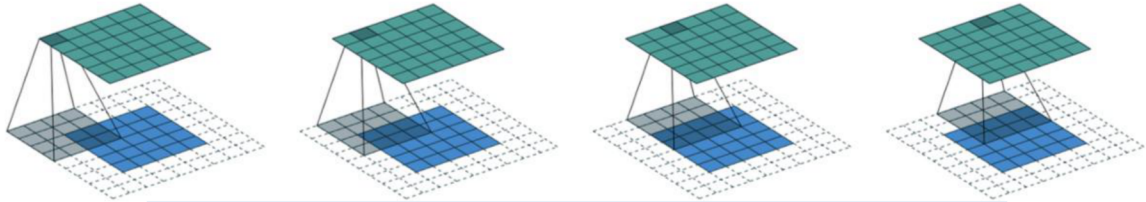
EE708

Module 7

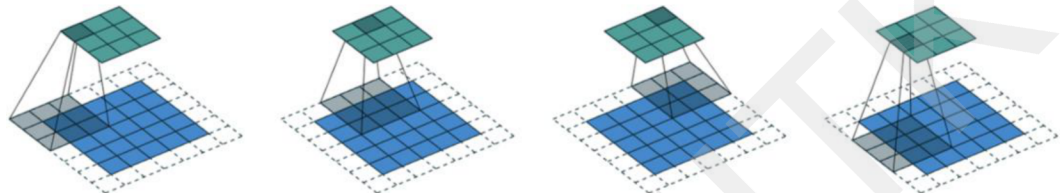
18

18

Padding and Strides



5 × 5 image with 2 pixels of padding convolved with 4 × 4 kernel and stride 1



5×5 image with 1 pixels of padding convolved with 3×3 kernel and stride 2

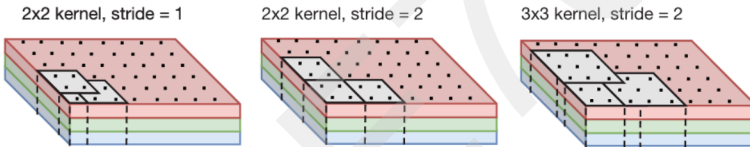
Source: Vincent Dumoulin and Francesco Visin
EE708

Module 7

19

19

Understanding Padding and Strides with Multiple Neurons



Examples of how receptive fields of three different neurons can overlap or adjoin. The image consists of 6 × 8 pixels.

	Kernel size	Stride	Number of neurons to cover full image	Observation
Example 1	2 × 2	1	5 × 7	Focus of each neuron is separated by only a single pixel
Example 2	2 × 2	2	3 × 4	The larger the stride, the fewer neurons are needed to cover the entire image.
Example 3	3 × 3	2	3 × 4	

- Kernel size and stride are orthogonal parameters, but they do interact.
 - Example: For 2 × 2 kernel with a stride of 3, some image pixels remain unconnected to any neuron.
- The number of neurons needed to cover the image is primarily affected by the stride.
- **Stride length should be less than kernel size, But need to define Tradeoff between the two**

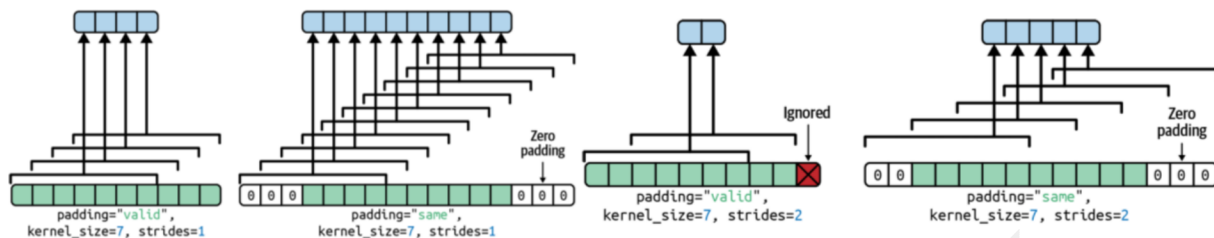
EE708

Module 7

20

20

Padding and Stride – Linear Illustration



When stride=1

With stride is greater than 1, the output is much smaller even when using "same" padding

EE708

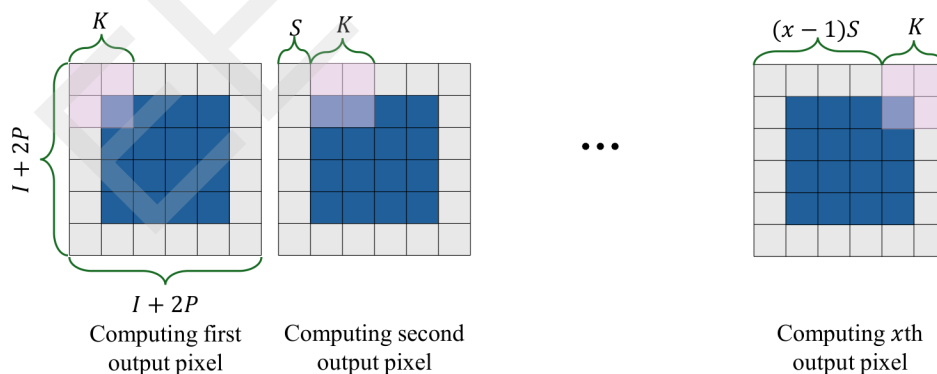
Module 7

21

21

Padding and Strides: Representation of Output Image

Let an image of size $I \times I$ convolved with a kernel of size $K \times K$ with stride S and padding of P .



EE708

Module 8

22

22

Padding and Strides: Representation of Output Image

- Side length in terms of K, x, S :

$$K + (x - 1)S$$

- Side length in terms of I and P :

$$I + 2P$$

- Equating both the above:

$$K + (x - 1)S = I + 2P$$

$$x = \left\lfloor \frac{I + 2P - K}{S} \right\rfloor + 1$$

- Example:** For convolution of 4×4 image with 3×3 kernel and stride 1, how much padding is required to obtain output feature size same as input size?

$$x = 4 \Rightarrow \left\lfloor \frac{I + 2P - K}{S} \right\rfloor + 1 = 4 \Rightarrow \left\lfloor \frac{4 + 2P - 3}{1} \right\rfloor + 1 \Rightarrow P = 1$$

EE708

Module 8

23

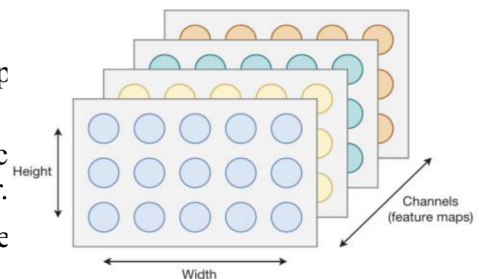
23

Convolutional Layers: Stacking Multiple Channels

- Image corresponds to a 2D data with width and height.
- Neurons are also grouped into channels or feature map in a third dimension.
- Neurons in the 3rd dimension are decoupled from each other and are together considered to form a single layer.
- Input images are composed of multiple sublayers: one per color channel.

- **3 for RGB** and **1 for grayscale images**.

- Weight sharing: All the neurons within a single channel have identical weights.
 - Dramatically reduces the number of parameters in the model.
 - Once CNN learns to recognize a pattern in one location, it can recognize it in any other location.
 - In contrast, once a fully connected NN has learned to recognize a pattern in one location, it can only recognize it in that particular location.



Topology of a 2D convolutional layer.

EE708

Module 7

24

24

Multichannel Convolution Operation

- A convolutional layer applies multiple kernels (or filters) to an input image, producing an output with a different number of channels.
- For RGB images (3 channels), the kernel size includes the input channels (e.g., $3 \times 3 \times 3$ for a 3×3 kernel).
- Kernels can specialize in detecting patterns.
- Each kernel operates on each channel of the image independently, multiplying corresponding elements and summing the results.
- Results from all filters across channels are summed to produce one value per grid location for each output feature.
- Steps for obtaining output:
 1. Take a kernel and convolve it for all input features.
 2. Take summation over the resulting convolution output from step 1.
 3. Repeat steps 1 and 2 for each kernel.

EE708

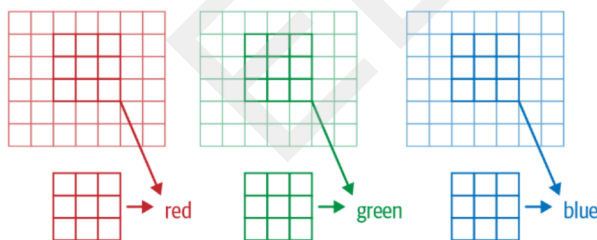
Module 7

25

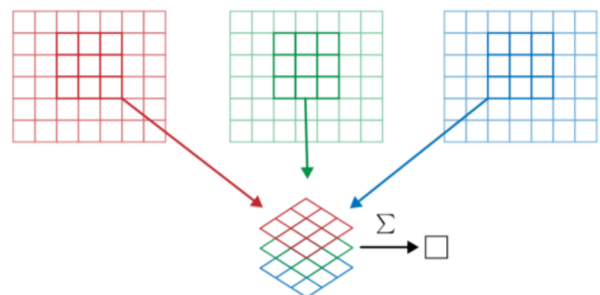
25

Multichannel Convolution Operation

- Example: In a color image, contributions from each channel (e.g., red, green, blue) are combined at each location to compute the output feature map.



Convolution over an RGB image



Adding the RGB filters

EE708

Module 7

26

26

-
- convolved with gives
- Input
 $F^{(\text{inp})}$ channels
 $\{\mathbf{X}^{(1)}, \mathbf{X}^{(1)}, \dots, \mathbf{X}^{(F^{(\text{inp})})}\}$
 $\mathbf{X}^{(i)}$ of size $I_h \times I_w$
- Kernel
 K kernels
 $\{\mathbf{W}^{(1)}, \mathbf{W}^{(1)}, \dots, \mathbf{W}^{(K)}\}$
 $\mathbf{W}^{(i)}$ of size $K_h \times K_w$
- Output
 $F^{(\text{out})}$ channels
 $\{\mathbf{O}^{(1)}, \mathbf{O}^{(1)}, \dots, \mathbf{O}^{(F^{(\text{out})})}\}$
 $\mathbf{O}^{(i)}$ of size $O_h \times O_w$

27

Multichannel Convolution Operation : More Detailed Illustration



14

Pooling Layers : Reducing Spatial Dimensions

- Pooling layer reduces spatial dimensions (height and width) of feature maps, lowering computation and preventing overfitting.
 - Each neuron in a pooling layer is connected to a small receptive field in the previous layer.
 - Size, stride, and padding type must be defined.
 - Works independently on every channel.
 - Pooling neuron has **no weights**, it aggregates the inputs using an aggregation function.
 - Some **spatial information is lost** during pooling.
- Types:
 - Max Pooling**: Select the maximum value from a region in the feature map.
 - Average Pooling**: Computes the average value of the region in a feature map.

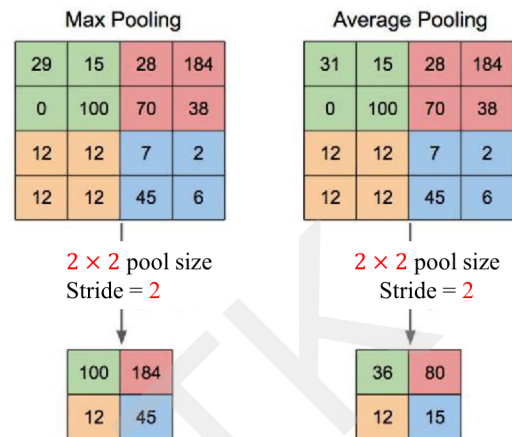


Illustration of max- and average-pooling with 8×8 input mapped to either the average or the max values for that patch.

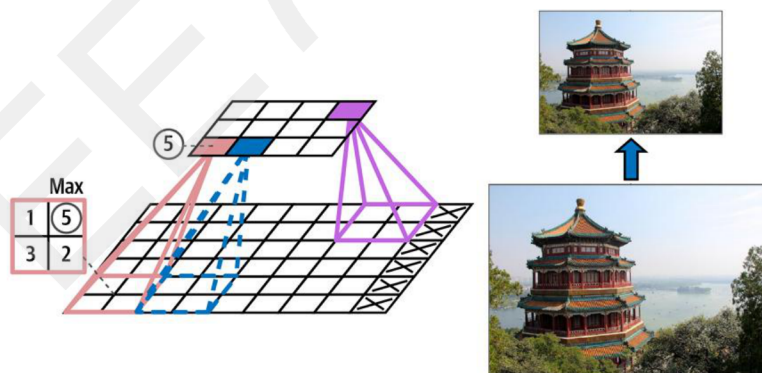
EE708

Module 7

29

29

Pooling Layers : Example Images



EE708

Module 7

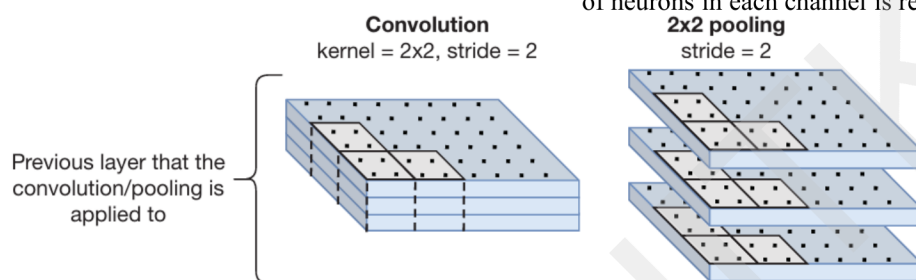
30

30

Pooling Layers: Max Pooling

1. Max pooling can be used as an alternative to convolution with a large stride.

- It reduces the size of a layer and is an alternative to a large stride.
- Convolution
 - With an F channel input and $K \times K$ size kernel, it combines $K \times K \times F$ neurons **across channels** to give an output.
 - Number of channels reduced to 1.
- Max pooling
 - With pooling layer of size $K \times K$, it combines $K \times K$ neurons **within a channel** to give an output.
 - Number of channels remain same but number of neurons in each channel is reduced.



EE708

Module 7

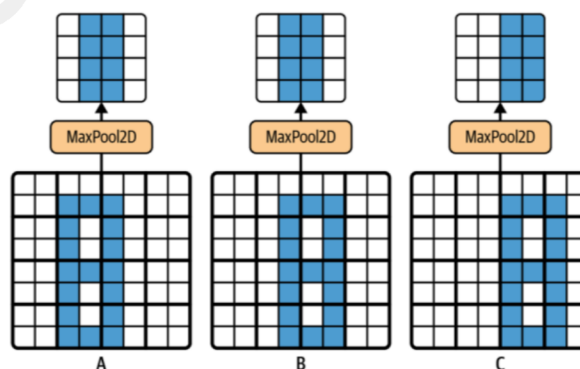
31

31

Pooling Layers: Max Pooling

2. Max pooling layer also introduces some **translational invariance**.

- Example:
 - Assume that the bright pixels have a lower value than dark pixels
 - Consider three images (A, B, C) with same information (digit 8) but are shifted by one and two pixels to the right.
 - Max pooling:
 - Outputs for images A and B are identical.
 - For image C, the output is different: it is shifted one pixel to the right (but there is still 50% invariance).



Max pooling 3 images (A, B, C) with a 2×2 kernel and stride 2

- **Inserting a max pooling layer every few layers in a CNN, it is possible to get some level of translation invariance at a larger scale.**

EE708

Module 7

32

32

Pooling Layers: Max Pooling

3. Max pooling offers a **small amount** of **rotational invariance** and **scale invariance**.
 - Such invariance can be useful in cases where the prediction should not depend on these details, such as in classification tasks.
 - Drawbacks:
 - Destructive: With a tiny 2×2 kernel and a stride of 2, the output is two times smaller in both directions (area is $4 \times$ smaller), dropping 75% of the input values.
 - In some applications, invariance is not desirable.
 - For example, in semantic segmentation (the task of labeling each pixel in an image with a class or category).
 - If the input image is translated by one pixel to the right, the output should be translated by one pixel to the right.
 - A small change to the inputs should lead to a corresponding small change in the output.

EE708

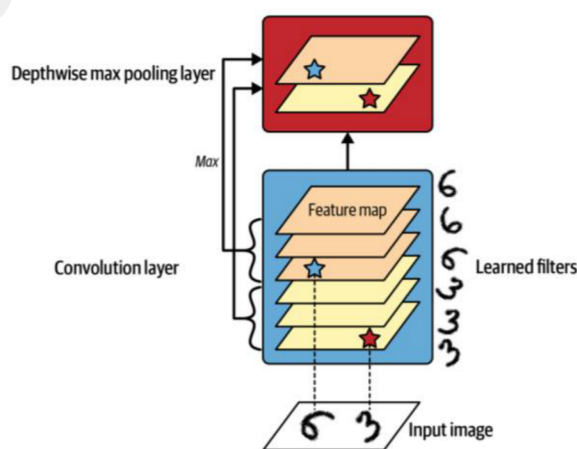
Module 7

33

33

Pooling Layers : Along Depth Dimension

- Max pooling and average pooling can be performed along the depth dimension instead of the spatial dimensions.
- This allows the CNN to learn to be invariant to various features.
- For example
 - Multiple filters where each filter detecting a different rotation of the same pattern.
 - Take **depthwise** max pooling layer across multiple filter.
 - This ensures that the output is the same regardless of the rotation.
- CNN could similarly learn to be invariant to anything: **thickness, brightness, skew, color**.



Depthwise max pooling can help the CNN learn to be invariant (to rotation in this case)

EE708

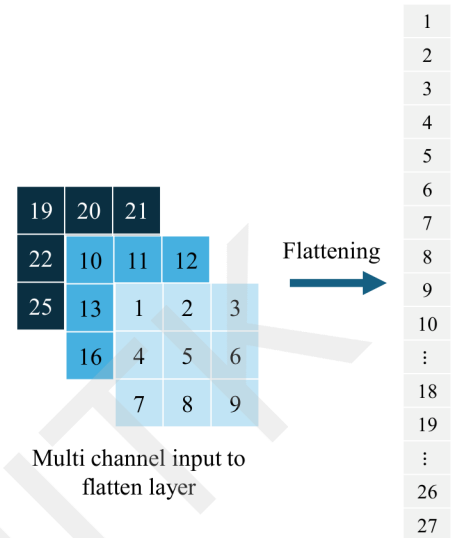
Module 7

34

34

Flatten Layer

- Transforms a multi-dimensional tensor into a 1D array.
- Bridges the transition between convolutional layers and fully connected layers.
- For an input to flatten layer of shape $I_h \times I_w \times I_d$, output is of size $I_h I_w I_d \times 1$.
- Flatten preserves all spatial and channel information as a vector but discards the spatial structure.
- The resulting vector is passed to fully connected layers for tasks such as classification or regression.



EE708

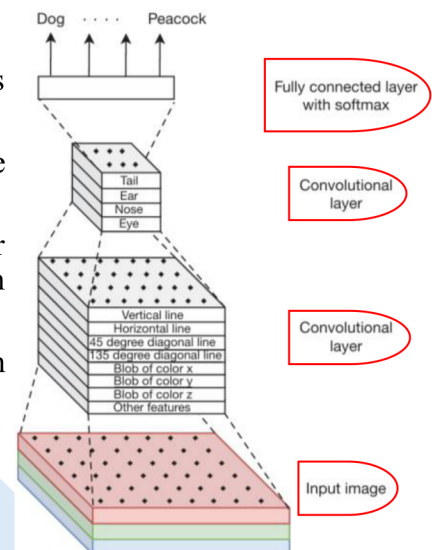
Module 7

35

35

Combining Convolutional and Fully Connected Layers into a Network

- First convolutional layer detects low-level features.
- Second convolutional layer combines low-level features into complex features.
- Fully connected layer with a softmax function classifies the image into one of C classes (e.g., dog or peacock).
- Neurons in the top convolutional layer have larger receptive fields compared to those in the bottom layer, even with the same kernel size.
- Larger receptive fields in higher layers enable the detection of more complex features.



Convolutional neural network built from two convolutional layers and one fully connected layer

EE708

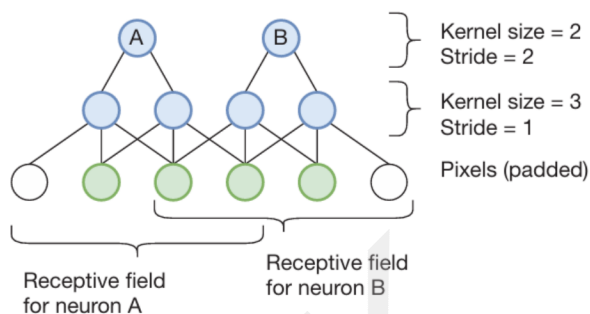
Module 7

36

36

Combining Convolutional and Fully Connected Layers into a Network

- Figure shows a 1D convolution and only a single channel in each convolutional layer.
- The input image comprises **four pixels** (green in the figure).
- The neurons in the first layer have a **kernel size of 3 and a stride of 1**. Resulting in feature maps of **size 4 (Or 4 Neurons)**
- The neurons in the second layer have a **kernel size of 2 and stride of 2**, Resulting in feature maps of **size 2 (Or 2 Neurons)**, but their **receptive fields are four pixels**.
- These two receptive fields overlap to some extent.
- Each neuron in the output layer summarizes more than half of the input image.



How the receptive field increases deeper into the network. Although the neurons in the topmost layer have a kernel size of only 2, their receptive fields are four pixels.

Note the padding of the input layer.

EE708

Module 7

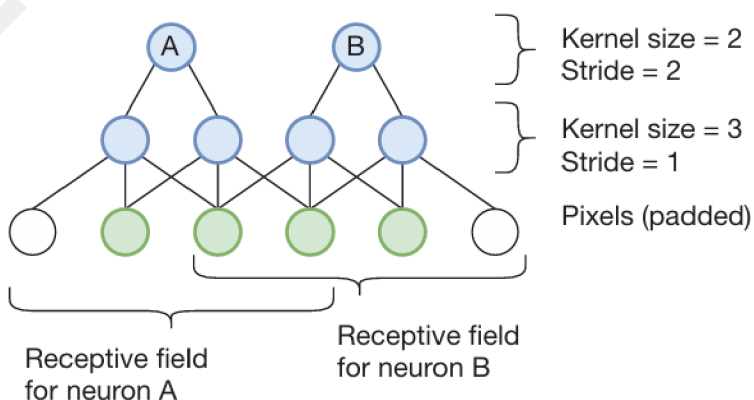
37

37

Combining Convolutional and Fully Connected Layers into a Network

Concept of padding the input image (white circles):

- Results in the same resolution as input resolution after the first convolution.
- It is applied to the input image and stacked convolutional layers.
- More critical in deeper network layers with smaller resolutions.
- Without padding, layer dimensions shrink after each convolution, regardless of stride.
- Helps maintain spatial dimensions, especially in deeper layers.



EE708

Module 7

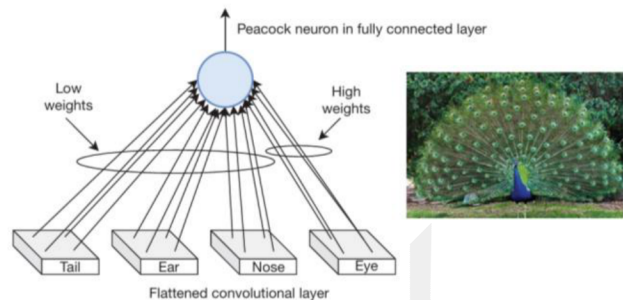
38

38

Combining Convolutional and Fully Connected Layers into a Network

How does a neuron classify the image as containing a peacock?

- In the fully connected layer, assign high weights to all neurons that represent eyes and low weights to most other neurons.
 - Based on the assumption that peacocks have many eye-like features.
- Potential Misclassification: It may confuse other objects (e.g., an alien with many eyes) for a peacock.
- However, similar errors could happen to humans in unexpected situations.



How a neuron in the fully connected layer combines multiple features into an animal classification.

EE708

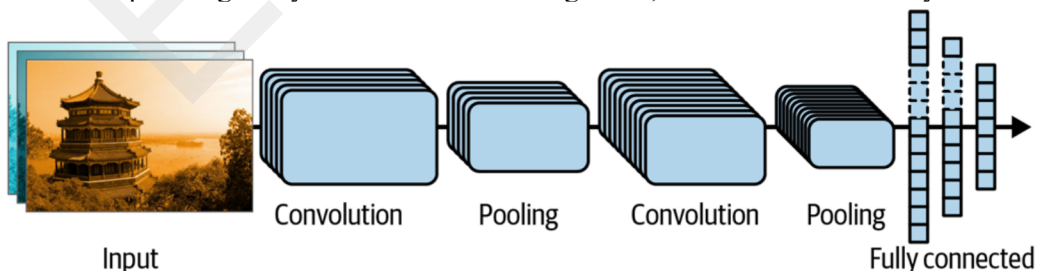
Module 7

39

39

CNN Architectures

- A common **mistake** is to **use convolution kernels that are too large**.
 - **Instead of using a convolutional layer with a 5 x 5 kernel, stacking two layers with 3 x 3 kernels reduces parameters and requires fewer computations, and it usually performs better.**
- One exception is for the first convolutional layer: it can typically have a large kernel, usually with a stride of 2 or more.
- This will reduce the spatial dimension of the image without losing too much information,
- Since the input image only has three channels in general, it will not be too costly.



Typical CNN architecture

EE708

Module 7

40

40

LeNet-5

- Developed by *Yann LeCun* in 1998.
- Most widely known CNN architecture.
- Widely used for handwritten digit recognition (MNIST).
- Main difference with more modern classification CNNs are the activation functions:
 - ReLU is used instead of tanh
 - softmax is used in output layer instead of RBF.

LeNet-5 architecture

Layer	Type	Maps	Size	Kernel size	Stride	Activation
Out	Fully connected	—	10	—	—	RBF
F6	Fully connected	—	84	—	—	tanh
C5	Convolution	120	1 × 1	5 × 5	1	tanh
S4	Avg pooling	16	5 × 5	2 × 2	2	tanh
C3	Convolution	16	10 × 10	5 × 5	1	tanh
S2	Avg pooling	6	14 × 14	2 × 2	2	tanh
C1	Convolution	6	28 × 28	5 × 5	1	tanh
In	Input	1	32 × 32	—	—	—

EE708

Module 7

41

41

AlexNet

- Developed by *Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012*
- Similar to LeNet-5, only much larger and deeper.
- The first architecture with stacked convolutional layers directly on top of one another, instead of stacking a pooling layer on top of each convolutional layer.
- To reduce overfitting, the authors used two regularization techniques.
 1. Applied dropout: 50% dropout rate during training to the outputs of layers F9 and F10.
 2. Data augmentation: randomly shifting the training images by various offsets, flipping them horizontally, and changing the lighting conditions.

AlexNet architecture

Layer	Type	Maps	Size	Kernel size	Stride	Padding	Activation
Out	Fully connected	—	1,000	—	—	—	Softmax
F10	Fully connected	—	4,096	—	—	—	ReLU
F9	Fully connected	—	4,096	—	—	—	ReLU
S8	Max pooling	256	6 × 6	3 × 3	2	valid	—
C7	Convolution	256	13 × 13	3 × 3	1	same	ReLU
C6	Convolution	384	13 × 13	3 × 3	1	same	ReLU
C5	Convolution	384	13 × 13	3 × 3	1	same	ReLU
S4	Max pooling	256	13 × 13	3 × 3	2	valid	—
C3	Convolution	256	27 × 27	5 × 5	1	same	ReLU
S2	Max pooling	96	27 × 27	3 × 3	2	valid	—
C1	Convolution	96	55 × 55	11 × 11	4	valid	ReLU
In	Input	3 (RGB)	227 × 227	—	—	—	—

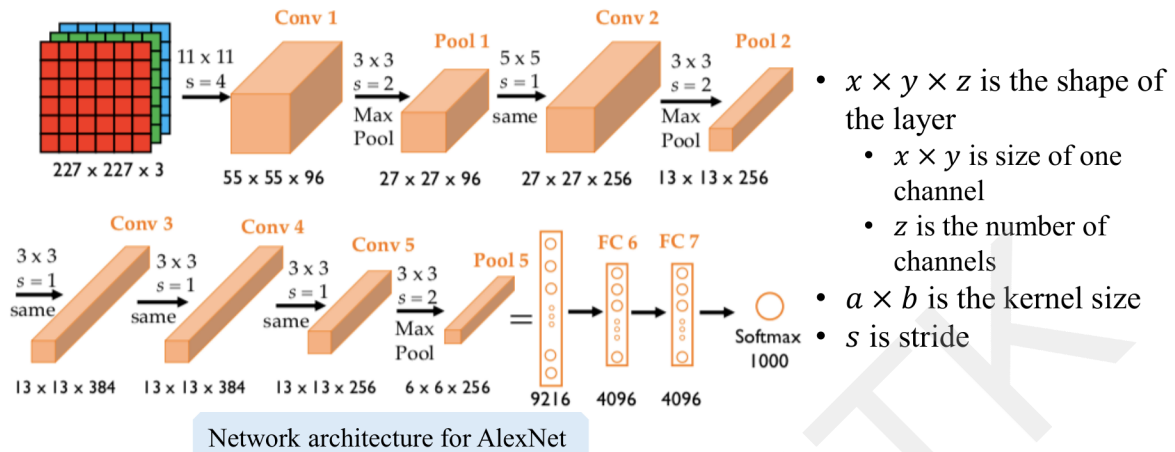
EE708

Module 7

42

42

AlexNet



Won the 2012 ILSVRC challenge by a large margin. Achieved a top-five error rate of 17%, while the second best competitor achieved only 26%.

EE708

Module 7

43

43

GoogLeNet

- Developed by *Christian Szegedy et al.* from Google Research.
- GoogLeNet features a significantly deeper network than earlier CNNs.
- Depth is achieved through **inception modules**, enabling more efficient parameter usage compared to previous architectures.
- Has 10 times fewer parameters than AlexNet (roughly 6 million instead of 60 million).
- Won the ILSVRC 2014 challenge by pushing the top-five error rate below 7%

EE708

Module 7

44

44

GoogLeNet

- The first two layers reduce image dimensions by a factor of 16 to lower computational cost, using a 7×7 kernel to retain information.
- **Local response normalization** encourages learning diverse features.
- **Max pooling layers** reduce height and width to speed up computations.
- **Nine inception** modules with interleaved max pooling layers for dimensionality reduction.
- **Global average pooling layer** computes the mean of each feature map, discarding remaining spatial details.
- Input images (224×224) are reduced to 7×7 after five max pooling layers. Avoiding multiple fully connected layers minimizes parameters and overfitting risk.
- Final layers include dropout, a fully connected layer with 1,000 units, and a softmax function for class probabilities.